

MLaD²: A Semi-Supervised Money Laundering Detection Framework Based on Decoupling Training

Xuejiao Luo[✉], Xiaohui Han[✉], *Member, IEEE*, Wenbo Zuo, Xiaoming Wu,
and Wenyin Liu[✉], *Senior Member, IEEE*

Abstract—Money laundering (ML) poses a severe threat to financial stability and social security. Various money laundering detection methods have emerged in the past two decades. Among these methods, some semi-supervised ones based on graph neural networks (GNNs) have achieved impressive performance. However, the homogeneity hypothesis of GNN-based methods does not fit the ML detection scenario, affecting the detection performance. This paper presents a semi-supervised money laundering detection framework based on decoupling training (MLaD²). MLaD² constructs a transaction relationship network based on node similarity (TRNNS) to model account interactions. Performing on TRNNS, MLaD² learns the representation of accounts using a GNN. The weighting mechanism of TRNNS can overcome the drawback of the homogeneity hypothesis. Based on the learned account representations, MLaD² adopts a decoupling training mechanism to build an ML accounts detection model, reducing its dependence on annotated data. The pre-training phase of the decoupling training employs a contrastive self-supervised learning model to learn the intrinsic characteristics of accounts. The fine-tuning phase extracts discriminative features between ML accounts and benign accounts with labeled data. Comprehensive evaluations and comparisons on a real-world ML dataset demonstrate that MLaD² yields results that surpass existing methods, especially when training with a small scale of labeled samples.

Index Terms—Money laundering detection, contrastive self-supervised learning, graph neural network.

I. INTRODUCTION

MONEY laundering (ML) refers to activities aimed at turning dirty money (i.e., funds obtained by illegal activities such as fraud, drug trafficking, and gambling) into lawful money [1]. It can feed back into these illegal activities, negatively affecting financial stability and social security. According to an International Monetary Fund survey, the amount of money laundered is 2% to 5% of global GDP [2]. Thus, ML detection techniques have attracted great attention from both industry and academia.

Early ML detection solutions employ a set of heuristic rules generated by human experts to identify suspicious transactions or accounts [3], [4]. The disadvantages of rule-based solutions are the high false-positive rate, the need for human resources, and the inability to recognize new ML patterns. More recent ML techniques have leveraged machine learning or deep learning models to improve detection accuracy and generalization while reducing reliance on expert knowledge. These techniques mainly fall into three categories: supervised, semi-supervised, and unsupervised methods.

Supervised methods [5], [6], [7], [8] need a large quantity of labeled data to train a classifier for money laundering identification. However, obtaining ground-truth labels for transactions is still labor-intensive. In contrast, unsupervised methods adopt strategies that do not require any ground-truth information, such as reconstruction error computation [9], graph density estimation [10], and outlier detection [11], [12]. However, these methods show unsatisfactory detection performance on real-world data due to inappropriate measurements of anomalies and the lack of supervision. Semi-supervised methods, which fall between supervised and unsupervised methods, use both a small amount of labeled data and a large amount of unlabeled data during training. A branch of semi-supervised methods, grounded in the principle of active learning [13], iteratively uses unsupervised learning to select a small-scale of data for expert annotation and then trains a supervised detection model with the annotated data [14]. Another branch of semi-supervised methods focuses on designing sophisticated

Manuscript received 14 August 2023; revised 20 December 2023; accepted 13 March 2024. Date of publication 25 March 2024; date of current version 6 May 2024. This work was supported in part by Shandong Provincial Natural Science Foundation of China under Grant ZR2022MF295 and Grant ZR2022MF257; in part by the Key Project of Quancheng Provincial Laboratory under Grant QCLZD202303; in part by the Fundamental Research Promotion Plan of Qilu University of Technology (Shandong Academy of Sciences) under Grant 2021JC02020; in part by the Pilot Project for Integrated Innovation of Science, Education and Industry of Qilu University of Technology (Shandong Academy of Sciences) under Grant 2022JBZ01-01; and in part by the Joint Open Project of Shandong Computer Society and Provincial Key Laboratory under Grant SKLCN2021-03. The associate editor coordinating the review of this manuscript and approving it for publication was Dr. Angelo Spognardi. (*Corresponding author: Xiaohui Han.*)

Xuejiao Luo, Xiaohui Han, Wenbo Zuo, and Xiaoming Wu are with the Key Laboratory of Computing Power Network and Information Security, Ministry of Education, Shandong Computer Science Center (National Supercomputer Center in Jinan), Qilu University of Technology (Shandong Academy of Sciences), Jinan 250014, China, and also with Shandong Provincial Key Laboratory of Computer Networks, Shandong Fundamental Research Center for Computer Science, Jinan 250353, China (e-mail: luoxj0116@gmail.com; xiaohhan@gmail.com; zuowbo@sdas.org; wuxm@sdas.org).

Wenyin Liu is with Zhongguancun Laboratory, Beijing 100086, China (e-mail: liuwuy@gdut.edu.cn).

Digital Object Identifier 10.1109/TIFS.2024.3380262

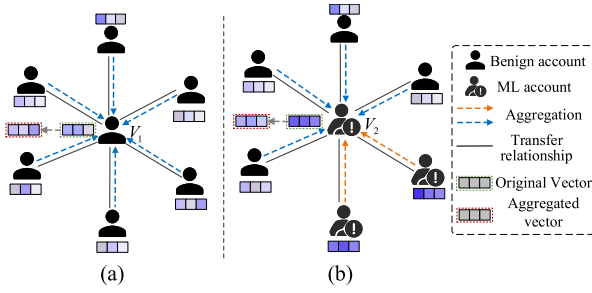


Fig. 1. Examples of the aggregation process on a transaction relationship network. (a) The aggregation process between a BA V_1 and its neighbors. All the neighbors of V_1 are also BAs. The representation vector of V_1 is updated by receiving the information from its neighbors. (b) The aggregation process between an MLA V_2 and its neighbors. The neighbors of V_2 contain both BAs and MLAs. The representation vector of V_2 after the aggregation looks similar to that of V_1 due to receiving information from its BA neighbors.

algorithms to accurately predict the categories of unlabeled samples using a small labeled dataset. Some recent methods in this branch based on graph neural networks (GNNs) have shown impressive detection performance.

GNN-based semi-supervised methods typically model the transaction relationships between bank accounts as a graph. They learn the representation of an account (i.e., a node in the graph) by aggregating information from its neighbors, including labeled and unlabeled samples. The learned representation determines whether the account is an ML participant [15], [16]. The underlying hypothesis is that a node in the graph shares similar characteristics and labels with its neighbors (i.e., the homogeneity hypothesis). For example, individuals in social networks may share similar interests with their friends; in citation networks, neighboring nodes (i.e., papers) usually share similar topics. Hence, information obtained from neighbors can enhance the representation of a node.

However, the homogeneity hypothesis does not fit the ML detection scenario. Through the analysis of our experimental dataset (see Section IV-A), we observed that ML participants often carry out transactions with both their accomplices and benign accounts (BAs) to hide the trace of ML. Some other studies have also observed this phenomenon [10], [17]. Therefore, in the transaction relation graph, the neighbors of an ML account (MLA) are composed of both MLAs and BAs. Under this condition, the aggregation mechanism of GNNs also brings the information of BA nodes into the representation of MLA, which is equivalent to introducing a type of noise. Moreover, the noise can dominate the aggregated information if BAs are more than MLAs. For example, in Figure 1, V_1 is a BA, whose neighbors are all BAs, while V_2 is an MLA, whose neighbors contain both BAs and MLAs. Before aggregation, the original representations of V_1 and V_2 are different. However, after aggregation, the representation of V_2 is influenced by its BA neighbors and becomes similar to those of V_1 .

To tackle the above issue, we propose a Semi-supervised Money Laundering Detection Framework Based on Decoupling Training (MLaD²) in this study. MLaD² constructs a Transaction Relationship Network based on Node Similarity (TRNNS) to model the interactions between accounts. TRNNS

is a weighted undirected graph, where nodes and edges represent accounts and money transfer relationships, respectively. An edge connecting two nodes means that money transfers have occurred between the corresponding two accounts. The weight of the edge is calculated based on the similarity between the two accounts. Performing on TRNNS, MLaD² learns the representations of accounts using a GNN. During the information aggregation process of GNN, the contribution of each neighbor to a node's representation is controlled according to the weight of the edge connecting them. This strategy can reduce the noise information from dissimilar neighbors. Based on the representations of accounts, MLaD² adopts a decoupling training mechanism [18] to build an MLA detection model. The decoupling training consists of a pre-training phase and a fine-tuning phase. In the pre-training phase, MLaD² learns the intrinsic features of accounts with a contrastive self-supervised learning model, which gains discriminative knowledge from the contrast of elaborate instance pairs instead of labels. Then, MLaD² further learns category features between MLAs and BAs with labeled data in the fine-tuning phase.

In summary, we make the following contributions to ML detection research.

- We propose a novel semi-supervised ML detection framework, MLaD². The novelty of MLaD² is that we introduce the principle of decoupling training in its learning process. Existing GNN-based semi-supervised methods learn node representations through a single training process. The training process takes ground-truth labels as supervision and performs information aggregation to update node representations iteratively. Unlike these methods, we decoupled the learning process of MLaD² into two phases. A pre-training phase learns node representations by iteratively performing information aggregation and contrastive self-supervised learning without the supervision of ground-truth labels. A fine-tuning phase further updates the node representations under the supervision of ground-truth labels. Benefiting from a decoupling training design, MLaD² can effectively leverage the discriminative knowledge in the intrinsic properties of accounts and the limited labeled data to mitigate the impacts of insufficient and imbalanced labeled samples.
- We reform GNN's information aggregation process by introducing a transaction relationship network TRNNS. In TRNNS, an edge connecting two nodes is weighted using their similarity. In the pre-training phase of MLaD², GNN information aggregations are performed on TRNNS. The contribution of each neighbor to a node's representation is differentiated based on the weight of the edge connecting them. As a result, MLaD² can selectively bring more information to a node from neighbors similar to it, reducing the noise aggregated from dissimilar neighbors. The reformed information aggregation process can overcome the drawback of the homogeneity hypothesis of GNN in the ML detection scenario.
- We introduce a novel type of contrastive instance pair, called "structure view versus behavior view" (SvB), for

contrastive self-supervised learning in the pre-training phase of MLaD². Conventional contrastive learning methods for graph data commonly use “node versus subgraph” or “graph versus graph” as instance pairs; each sample has a single information view. Since MLaD² considers an account’s behavior and structure features, SvB uses two different views to describe these two aspects of information. The two views of the same account are considered a positive instance pair, while two views of different accounts are considered a negative instance pair. By performing contrastive self-supervised learning with SvB pairs, MLaD² can learn fine-grained discriminative information between MLAs and BAs, reducing its dependency on labeled training data.

Comprehensive evaluations and comparisons on a real-world ML dataset demonstrate that MLaD² yields results that surpass existing methods, especially when using a small scale of labeled samples.

The rest of this paper is organized as follows: Section II briefly reviews the related work. Section III presents the details of MLaD². Section IV presents and analyzes our experimental results. Section V discusses the limitations of MLaD². Finally, Section VI concludes this paper.

II. RELATED WORK

A. Money Laundering Detection

ML detection technology has made significant progress in the past two decades. Early ML detection techniques are generally rule-based, which screen suspicious entities using rules (e.g., a threshold on the amount of money transferred) predefined by domain experts [6]. Rule-based techniques can accurately identify known ML patterns. However, they have disadvantages like high investigation costs and false positives. Moreover, money launderers can easily bypass the detection using new ML patterns.

Recent ML research has increasingly employed machine learning and deep learning models. Several works consider ML detection a supervised learning problem and build classifiers to identify ML transactions or accounts. Alkhalili et al. [19] took statistics of money transfer behavior (e.g., the number and amount of money transferred) as features and trained support vector machine (SVM), decision tree (DT), and naive bayesian (NB) as ML recognizers. Gu et al. [20] used a long short-term memory (LSTM) network to build an anomalous transaction detection model on the time sequences of behavior statistics. Similarly, Jensen and Iosifidis [21] built an ML detection model using a bidirectional gated recurrent unit (bi-GRU) instead of LSTM. Some studies combine account profile attributes (e.g., gender, location, age, and the number of used services) with behavior statistics as features to build classifiers [17], [22], [23], [24]. Besides, the structural importance metrics (e.g., degree, betweenness, closeness, and transitivity) of an account in a money transfer relationship network are also commonly employed as features [5], [25], [26]. For example, Jullum et al. [26] used an XGBoost to alarm ML entities based on behavior statistics, structural metrics, and account profiles.

Supervised ML detection approaches need a large number of labeled training samples to achieve satisfactory performance. However, acquiring such a quantity of labeled data is typically impossible due to the high labor cost. To tackle this issue, some researchers built ML detection models based on unsupervised learning. Chen et al. [9] designed an automatic encoder (AE) and used the reconstruction error of transaction statistics to identify ML transactions. Li et al. [10] detected MLAs by searching dense money transfer flows. Sun et al. [11] used the statistical deviation of account balances to recognize MLAs. Lee et al. [12] detected the “smurfing” pattern of ML by minimizing the minimum adjacency matrix description length [27] of the adjacency matrix of a time-evolving accounting graph. Although unsupervised ML detection approaches do not need labeled data, their detection performance on real-world data is unsatisfactory due to the lack of supervision. In addition, finding an appropriate measurement of anomalies to identify ML transactions or accounts is typically tricky.

To leverage the strengths of supervised and unsupervised approaches and mitigate their weaknesses, many studies designed ML detection models based on semi-supervised learning paradigms, which utilize labeled and unlabeled data for training. A branch of these studies adopts active learning to select a small-scale of the most informative samples for labeling iteratively [13]. For example, Labanca et al. [14] proposed an active learning framework for ML detection that leverages an unsupervised model to select the most suspicious transactions for expert investigation and trains a supervised ML detection model on experts’ feedback (i.e., the labeled dataset). One drawback of active learning-based methods is that their entire training process requires expert involvement. Another branch of semi-supervised methods explores ingenious algorithms to accurately predict the categories of unlabeled samples with a small scale of labeled data. Several methods based on GNNs in this branch have shown impressive detection performance. Weber et al. [15] proposed an MLA detection approach based on the evolved graph convolutional neural network (EvolveGCN). Assumpção et al. [28] leveraged GraphSAGE [29] and a multilayer perceptron (MLP) to generate embeddings of nodes and edges in a money transfer network to detect MLAs. Most existing semi-supervised attempts employ GNNs or graph embedding-based models as their base model, aggregating neighborhood information to learn node embeddings. However, these approaches ignore the inconsistency problem between a node and its neighbors in ML scenarios (see Figure 1), which hinders their ability to capture anomalous information of MLAs effectively. In contrast, MLaD² uses a novel TRNNS to address the above issue. It can assign greater weight to the information of neighbors exhibiting high similarity with the target node.

Another characteristic of MLaD² is that it decouples the training process. Existing semi-supervised methods jointly learn node representations and an ML detector in a unified process. MLaD² decouples the learning of node intrinsic features and the learning of the MLA detector into two separate phases, which can reduce the dependency on labeled training data and mitigate sensitivity to class imbalance [30].

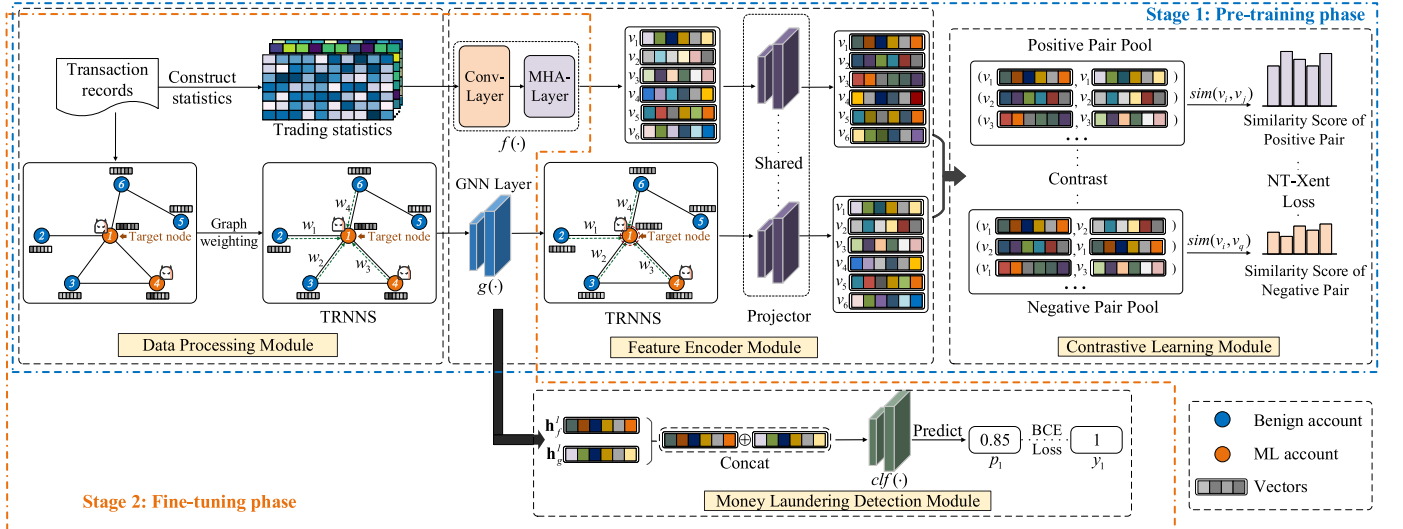


Fig. 2. Illustration of MLAD². MLAD² consists of four modules, i.e., the data processing module, the feature encoder module, the contrastive learning module, and the money laundering detection module. The MLAD² workflow has two phases, i.e., the pre-training phase and the fine-tuning phase.

B. Contrastive Self-Supervised Learning

Self-supervised learning (SSL) is a type of unsupervised learning, and contrastive self-supervised learning (CSSL) is an essential branch of SSL. CSSL learns data representations by discriminating positive and negative instance pairs [18]. It has dramatically progressed in visual processing methods, such as MoCo [31] and SimCLR [32]. Some recent works have extended CSSL to GNNs. Velivckovic et al. [33] took a node's representation and a graph's representation as a contrastive instance pair (i.e., node versus subgraph) and generated negative instances by graph corruption. Wang et al. [34] suggested an anomaly detection framework DCI. DCI treats the feature representation of a node as the positive sample and its row-wise shuffled version as the negative sample. Besides, Lo et al. [35] and Zhou et al. [36] also performed graph-level CSSL and then used a linear layer or an RF to detect suspicious entities.

The above methods use “node versus subgraph” or “graph versus graph” as instance pairs. They generate an embedding of a graph for contrastive learning by averaging the values of all node representations in the graph. However, the averaging operation may result in a loss of fine-grained characteristics of nodes in the graph, making it challenging to learn discriminative features. Unlike these methods, MLAD² uses a novel type of instance pair (i.e., SvB) for CSSL. It focuses more on the fine-grained discriminative characteristics between nodes. As a result, MLAD² exhibits a more excellent capability in detecting inconspicuous suspicious features within accounts.

III. PROPOSED APPROACH

Figure 2 shows the framework of MLAD², which includes four components: a data processing module, a feature encoder module, a contrastive learning module, and a money laundering detection module. The data processing module processes the raw transaction records to calculate transaction behavior

TABLE I
NOTATIONS

Notation	Description
R	A transaction record.
$N; T$	The # of (accounts; time slices).
$\mathbf{M}_i$	Transaction statistics of account i during T .
G	A TRNNS.
$\mathcal{V}; \mathcal{E}$	The node and edge set in the TRNNS.
$f(\cdot); \varphi$	The behavior feature encoder and its parameters.
$g(\cdot); \theta$	The structure feature encoder and its parameters.
$Projector(\cdot); \zeta$	The feature projector and its parameters.
$clf(\cdot); \xi$	The ML detector and its parameters.
$\mathbf{H}_f; \mathbf{H}_g$	Embeddings of (behavior view; structure view).
$\mathbf{E}_f; \mathbf{E}_g$	Linear projection embeddings of $\mathbf{H}_f$ and $\mathbf{H}_g$.
$\mathcal{L}_{SSL}; \mathcal{L}_{SL}$	Objective function of (pre-training; fine-tuning).
$\parallel$	The concatenate operation.
$\mathcal{N}_{(u_i)}$	1-order neighbor set of account u_i .
$\mathcal{D}_{ot}; \mathcal{D}_{tt}$	The original and tampered test data, respectively.

statistics and build TRNNS. The *feature encoder module* generates behavior and structure views for each account based on the statistics and TRNNS. The *contrastive learning module* leverages the behavior and structure view information to construct instance pairs and learns discriminative features by contrasting their corresponding embeddings. A pre-training process based on NT-Xent loss guides the representations of the two views of the same account to fall into nearby locations in the embedding space. After the pre-training, the *money laundering detection module* combines the behavior view embedding with the structure view embedding and then predicts the category of the corresponding node. In the following sections, we describe the details of each module. Table I lists the notations used in this paper.

A. Data Processing Module

The *data processing module* constructs behavior statistics and TRNNS based on the input transaction records.

TABLE II
DEFINITION OF STATISTICAL VARIABLES

Symbol	Feature Notation	Definition	Symbol	Feature Notation	Definition
s_1	Total amount of received money	$\sum_i a_{in}^i$, where a represents the amount of a transfer. For simplicity.	s_{17}	Max. # of transactions	$\text{Max}(\sum_i \sum_{j \in \mathcal{N}_{(i)}} \text{trans}_{ij})$
s_2	# of incoming clients	$\sum_i \sum_{j \in \mathcal{N}_{(i)} \& \text{Dst}(j)=i} u_{ij}$, where u represents counterparties of account i .	s_{18}	# of turnover with integer value	$\sum_i \sum_k (a^i \% 1 == 0)$, where K is the total # of transfer.
s_3	# of received	$\sum_i \sum_{j \in \mathcal{N}_{(i)} \& \text{Dst}(j)=i} \text{trans}_{ij}$, where trans_{ij} represents existing a transaction between i and j .	s_{19}	Pct. of turnover with integer values	$\sum_i (\sum_k (a^i \% 1 == 0)) / \sum_{j \in \mathcal{N}_{(i)}} \text{trans}_{ij}$
s_4	Avg. amount of received	$\text{Amt}_{in}^i / \sum_i \sum_{j \in \mathcal{N}_{(i)} \& \text{Dst}(j)=i} \text{trans}_{ij}$	s_{20}	Avg. amount of each transaction	$s_{13} / \sum_i \sum_{j \in \mathcal{N}_{(i)}} \text{trans}_{ij}$
s_5	Var. amount of received	$\sum_i ((a_{in}^i - \mu_{\text{Amt}_{in}})^2 / \sum_{j \in \mathcal{N}_{(i)} \& \text{Dst}(j)=i} \text{trans}_{ij})$	s_{21}	Pct. of # of payments to # of transactions	$\sum_i (\sum_{j \in \mathcal{N}_{(i)} \& \text{Src}(j)=i} \text{trans}_{ij} / \sum_{j \in \mathcal{N}_{(i)}} \text{trans}_{ij})$
s_6	Avg. amount of each incoming clients	$s_1 / \sum_i \sum_{j \in \mathcal{N}_{(i)} \& \text{Dst}(j)=i} u_{ij}$	s_{22}	Pct. of # of received to # of transactions	$\sum_i (\sum_{j \in \mathcal{N}_{(i)} \& \text{Dst}(j)=i} \text{trans}_{ij} / \sum_{j \in \mathcal{N}_{(i)}} \text{trans}_{ij})$
s_7	Total amount of payment	$\sum_i a_{out}^i$	s_{23}	Net income	$s_1 - s_7$
s_8	# of receiver	$\sum_i \sum_{j \in \mathcal{N}_{(i)} \& \text{Src}(j)=i} u_{ij}$	s_{24}	Pct. of receivers to counterparties	$\sum_i (\sum_{j \in \mathcal{N}_{(i)} \& \text{Src}(j)=i} u_{ij} / \sum_{j \in \mathcal{N}_{(i)}} u_{ij})$
s_9	# of payments	$\sum_i \sum_{j \in \mathcal{N}_{(i)} \& \text{Dst}(j)=i} \text{trans}_{ij}$	m_{25}	Var. amount of transaction	$s_{13} / \sum_i \sum_{j \in \mathcal{N}_{(i)}} \text{trans}_{ij}$
s_{10}	Avg. amount of payment	$s_7 / \sum_i \sum_{j \in \mathcal{N}_{(i)} \& \text{Src}(j)=i} \text{trans}_{ij}$	s_{26}	Avg. interval between transactions	$\sum_i (\sum_{t=t_{end}}^t (\text{trans}_t^i - \text{trans}_{t-1}^i) / \sum_{j \in \mathcal{N}_{(i)}} \text{trans}_{ij} - 1)$, where t is the time that the transaction occurred.
s_{11}	Var. amount of payment	$\sum_i ((a_{out}^i - s_{10})^2 / \sum_{j \in \mathcal{N}_{(i)} \& \text{Src}(j)=i} \text{trans}_{ij})$	s_{27}	Var. interval between transactions	$\sum_i (\sum_{t=t_{end}}^t (\text{trans}_t^i - \text{trans}_{t-1}^i - \mu_t)^2 / (\sum_{j \in \mathcal{N}_{(i)}} \text{trans}_{ij} - 1))$
s_{12}	Avg. amount of each receiver	$s_7 / \sum_i \sum_{j \in \mathcal{N}_{(i)} \& \text{Src}(j)=i} u_{ij}$	s_{28}	Pct. of night turnover	s_{13}^* / s_{13}
s_{13}	Turnover	$\sum_i (a_{in}^i + a_{out}^i)$	s_{29}	# of night transactions	$\sum_{i=1} \sum_{j \in \mathcal{N}_{(i)}} \text{trans}_{ij}$
s_{14}	# of counterparties	$\sum_i \sum_{j \in \mathcal{N}_{(i)}} u_{ij}$	s_{30}	Avg. amount of night turnover	$s_{13}^* / \sum_i \sum_{j \in \mathcal{N}_{(i)}} \text{trans}_{ij}$
s_{15}	Total # of transactions	$\sum_i \sum_{j \in \mathcal{N}_{(i)}} \text{trans}_{ij}$	s_{31}	Avg. the interval between transactions during the night	$\sum_i (\sum_{t=t_{end}}^t (\text{trans}_t^{i*} - \text{trans}_{t-1}^{i*}) / \sum_{j \in \mathcal{N}_{(i)}} \text{trans}_{ij} - 1)$
s_{16}	Pct. of Net income	$(s_1 - s_7) / (s_1 + s_7)$	s_{32}	Var. the interval between transactions during the night	$\sum_i (\sum_{t=t_{end}}^t (\text{trans}_t^{i*} - \text{trans}_{t-1}^{i*} - \mu_{t*})^2 / (\sum_{j \in \mathcal{N}_{(i)}} \text{trans}_{ij} - 1))$, where μ_{t*} is the Avg. interval between night transactions.

t^* and $*$ mean at night (i.e., to the time from 22:00 p.m. to 5:00 a.m. the next day).

$\text{Src}(i)$ means a set of accounts that transfer money to account i , $\text{Dst}(i)$ means account i transfers money to a set of accounts.

Pct.=percent.

1) *Transaction Behavior Statistics Calculation*: Let $R = [u_{src}, u_{dst}, time, amount]$ denote a transaction record, where u_{src} , u_{dst} , $time$, and $amount$ represent the sender, receiver, timestamp, and transaction amount, respectively. We divide the timeline of R into T slices with a 24-hour granularity and design 32 statistical variables (i.e., $\{s_1, s_2, \dots, s_{32}\}$) to track the transfer behavior characteristics of accounts. Table II lists the statistical variables, the efficacy of which has been demonstrated in our previous work [37]. For account u_i , let $\{\mathbf{M}_i\}_{i=1}^N \in \mathbb{R}^{d \times T}$ denote the transaction statistics during T slices, where N is the number of accounts, and d is the number of statistical variables.

2) *TRNNS Building*: The basis of the aggregation mechanism of GNN is the homogeneity hypothesis, which considers connected nodes in a graph to have similar characteristics. Since the neighbors of an MLA typically contain both MLAs and BAs, we should reduce the information from dissimilar neighbors during the aggregation process. One possible solution is to use the graph attention network (GAT) [38] to compute weights for each node, but this method has two issues: (1) GAT is sensitive to parameter initialization, and (2) the effect of GAT is negligible or even harmful on large and noisy graphs [39]. To address this issue, we build a TRNNS to weigh the relationship between nodes. Let $G = \{\mathcal{V}, \mathcal{E}\}$ denote the TRNNS, where $\mathcal{V} = \{v_1, v_2, \dots, v_N\}$ is the set of nodes representing accounts, and $\mathcal{E} = \{< i, j > | i, j \in \mathcal{V}\}$ is the set of edges representing transfer interactions between accounts. Each edge in $\mathcal{E}$ is weighted by the similarity scores between

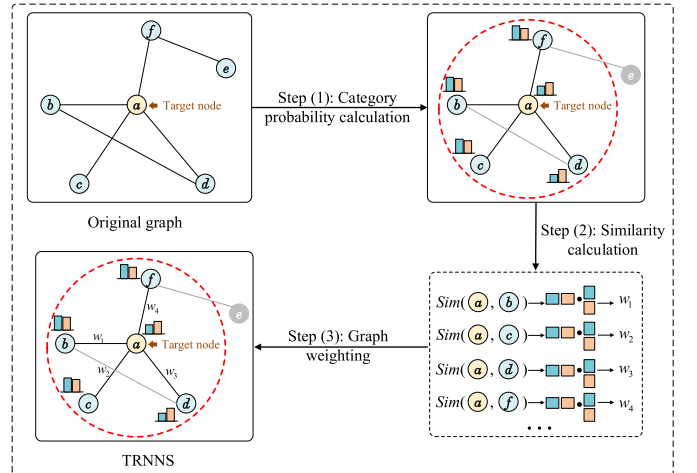


Fig. 3. Diagram of the TRNNS construction process. Building a TRNNS involves three steps: (1) category probability calculation, (2) similarity calculation, and (3) graph weighting. $\text{Sim}(\cdot)$ and w denote the similarity function and the weight of edges, respectively.

the two connected nodes. The construction of TRNNS consists of three steps, as shown in Figure 3.

Step (1): Category Probability Calculation: Let u_i be an unlabeled node in the TRNNS (e.g., the “node a ” in Figure 3). We denote its 1-order neighbors as $\mathcal{N}_{(u_i)} = \{v_i^1, v_i^2, \dots, v_i^n\}$, where n is the number of u_i ’s 1-order neighbors. The probability of u_i belonging to class c_k is:

$$P(c_k | u_i) \propto P(c_k | v_i^1) P(c_k | v_i^2) \cdots P(c_k | v_i^n). \quad (1)$$

According to conditional probability, $P(c_k | v_i^j)$ can be further decomposed as:

$$P(c_k | v_i^j) = \frac{P(v_i^j | c_k)P(c_k)}{P(v_i^j)}, j \in [1, n]. \quad (2)$$

Thus, $P(c_k | u_i)$ can be converted into the following formula:

$$P(c_k | u_i) \propto \frac{P(v_i^1 | c_k)P(v_i^2 | c_k) \cdots P(v_i^n | c_k)P(c_k)^n}{P(v_i^1)P(v_i^2) \cdots P(v_i^n)}, \quad (3)$$

$$\propto \prod_{j=1}^n P(v_i^j | c_k)P(c_k)^n.$$

$P(c_k)$ and $P(v_i^j | c_k)$ can be obtained as follows:

$$P(c_k) = \frac{\sum_{i=1}^{N_L} I(y_i = c_k)}{N_L}, k \in \{0, 1\}, \quad (4)$$

$$P(v_i^j | c_k) = \frac{\sum_{i=1}^{N_L} I(\mathcal{N}_{(u_i)}^j = v_i^j, y_i = c_k)}{\sum_{i=1}^{N_L} I(y_i = c_k)}, \quad (5)$$

where N_L is the number of labeled nodes, $I(\cdot)$ is an indicator function, and $\mathcal{N}_{(u_i)}^j$ represents the j -th neighbor of u_i . Finally, the probability of u_i belonging to class c_k can be calculated as:

$$P(c_k | u_i) = \frac{P(c_k)^n \prod_{j=1}^n P(v_i^j | c_k)}{\sum_{k=\{1,2\}} P(c_k)^n \prod_{j=1}^n P(v_i^j | c_k)}. \quad (6)$$

The class probability distribution of all nodes forms a matrix $\mathbf{P} \in \mathbb{R}^{N \times 2}$, in which the i -th row vector $\mathbf{p}_i$ is the class probability distribution of u_i . The labeled node's class probability distribution is according to its label information. For example, if a node is an MLA, its probability of being anomalous will be 1, while the probability of it being benign will be 0.

Step (2): Similarity Calculation: Dot-product is a simple and effective operator to measure the distance between two vectors, and has been demonstrated to be effective in link prediction tasks. Thus, we perform a dot-product on $\mathbf{P}$ to measure the degree of similarity between two arbitrary accounts:

$$\mathbf{W}_p = \mathbf{P} \cdot \mathbf{P}^\top, \quad (7)$$

where $\mathbf{W}_p \in \mathbb{R}^{N \times N}$ is the similarity score of N accounts.

Step (3): Graph Weighting: We weigh edges between nodes based on the similarity scores calculated in Step (2) and finally obtain the well-built TRNNS G .

B. Feature Encoder Module

As shown in the upper middle of Figure 2, we use a *behavior feature encoder* $f(\cdot)$, a *structure feature encoder* $g(\cdot)$, and a *feature projector* to obtain the behavior and structure views of each account.

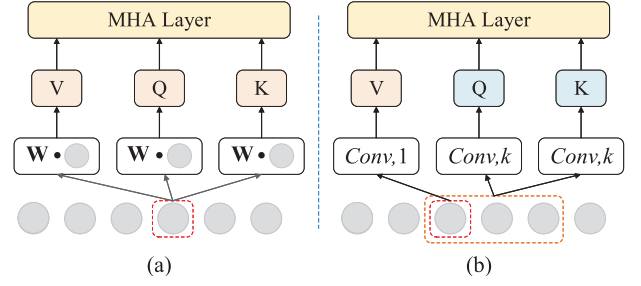


Fig. 4. Comparison between traditional Transformer and convolutional-based Transformer. (a) shows the architecture of traditional Transformer, where “ $\mathbf{W}$ ” denotes the weight parameter matrix. (b) is the architecture of convolutional-based Transformer, where “ $\text{Conv}, 1$ ” and “ Conv, k ” represent convolution of kernel size 1×1 and $k \times 1$ with stride 1, respectively. “MHA Layer” represents the multi-head self-attention layer.

1) Behavior Feature Encoder: The target of the *behavior feature encoder* is to capture both the long and short-term dependencies of transaction behavior. Recurrent neural networks (RNNs), such as LSTM [40] and GRU [41], have been widely employed to capture the long-term characteristics of time series in many scenarios. However, they suffer from the problems of vanishing and exploding gradients and cannot handle long sequences very well. Some studies have demonstrated that Transformer [42] performs much better than LSTM in capturing fairly long-term dependencies [43]. Thus, we adopt a Transformer-based approach to capture long-term patterns.

Specifically, it contains two multi-head self-attention (MHA for abbreviation) layers. The MHA layers generate N_H distinct query, key, and value matrices from $\mathbf{M}_i$, where N_H denotes the number of attention heads. However, as shown in Figure 4 (a), the traditional Transformer model calculates the similarity between queries and keys based on their pointwise values, resulting in the losing of short-term features. To address this issue, we apply convolution on input values before the attention layers to capture short-term features, as shown in Figure 4 (b). More specifically, we use a 3×1 convolution kernel with a stride of 1 on $\mathbf{M}_i$ to obtain the query matrix and the key matrix, respectively:

$$\mathbf{H}_Q^{N_{h_i}} = (\tanh(\mathbf{M}_i \otimes \mathbf{W}_{conv}^Q + \mathbf{b}_Q))\mathbf{W}_Q, \quad (8)$$

$$\mathbf{H}_K^{N_{h_i}} = (\tanh(\mathbf{M}_i \otimes \mathbf{W}_{conv}^K + \mathbf{b}_K))\mathbf{W}_K, \quad (9)$$

where $\mathbf{H}_Q^{N_{h_i}}, \mathbf{H}_K^{N_{h_i}} \in \mathbb{R}^{T \times \frac{d_c}{N_H}}$, d_c is the output dimension of convolution, $\otimes$ denotes the convolutional operation, and N_{h_i} is the h_i -th attention head. The value matrix is calculated by performing a convolution of kernel size 1×1 with stride 1 on $\mathbf{M}_i$:

$$\mathbf{H}_V^{N_{h_i}} = (\tanh(\mathbf{M}_i \otimes \mathbf{W}_{conv}^V + \mathbf{b}_V))\mathbf{W}_V, \quad (10)$$

where $\mathbf{H}_V^{N_{h_i}} \in \mathbb{R}^{T \times \frac{d_c}{N_H}}$. After these convolutional projections, the scaled dot-product attention computes the final outputs:

$$\mathbf{H}_a^{N_{h_i}} = \text{softmax}\left(\frac{\mathbf{H}_Q^{N_{h_i}} \mathbf{H}_K^{N_{h_i}}}{\sqrt{d_c}}\right)\mathbf{H}_V^{N_{h_i}}. \quad (11)$$

In practice, we use two attention headers, i.e., $N_H = 2$. Then, we concatenate $\mathbf{H}_a^{N_{h1}}$ and $\mathbf{H}_a^{N_{h2}}$ and perform a linear projection:

$$\mathbf{H}_f = [\mathbf{H}_a^{N_{h1}} \parallel \mathbf{H}_a^{N_{h2}}] \mathbf{W}_l + \mathbf{b}_f. \quad (12)$$

By computing the similarity of queries and keys from their local information rather than pointwise values, the behavior feature encoder obtains the transaction behavior embeddings $\mathbf{H}_f \in \mathbb{R}^{T \times d_c}$ with both long and short-term patterns.

2) *Structure Feature Encoder*: This component aims to aggregate neighbor information to enhance the representation of a node. We input the TRNNS G into a GNN with L stacking layers to reduce the noise information from dissimilar neighbors. The l -th layer can be computed as follows:

$$\mathbf{H}_g^l = GNN(\mathbf{A}, \mathbf{H}_g^{l-1}), \quad (13)$$

where $\mathbf{H}_g^l \in \mathbb{R}^{N \times d_g}$ is the nodes' embedding of G learned by the l -th layer, d_g is the embedding size of this layer, and $\mathbf{A} \in \mathbb{R}^{N \times N}$ is the adjacency matrix of G . Similarly, $\mathbf{H}_g^{l-1} \in \mathbb{R}^{N \times d_g}$ is the embedding of nodes learned by the $(l-1)$ -th layer. We initialize $\mathbf{H}_g^l$ with $\mathbf{X}$ when $l = 1$, where $\mathbf{X} \in \mathbb{R}^{N \times d}$ is the average values of d variables of T time slices. Since GCN is efficient and effective for the node classification task [44], we adopt a two-layer GCN as the GNN base model in practice. Thus, Eq.(13) can be specifically formulated as

$$\mathbf{H}_g = \hat{\mathbf{A}} \text{ReLU}(\hat{\mathbf{A}} \mathbf{X} \mathbf{W}_{g1}) \mathbf{W}_{g2}, \quad (14)$$

where $\hat{\mathbf{A}} = \tilde{\mathbf{D}}^{-\frac{1}{2}} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-\frac{1}{2}}$. $\tilde{\mathbf{A}}$ is the adjacency matrix with self-connections, i.e., $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}_N$. $\mathbf{I}_N \in \mathbb{R}^{N \times N}$ represents an identity matrix. $\tilde{\mathbf{D}} \in \mathbb{R}^{N \times N}$ is the degree matrix with $\tilde{\mathbf{D}}_{ii} = \sum_j \tilde{\mathbf{A}}_{ij}$. $\mathbf{W}_{g1}$ and $\mathbf{W}_{g2}$ are the learnable weight matrices of the 1-st and 2-nd graph convolutional layers, respectively. The output $\mathbf{H}_g \in \mathbb{R}^{N \times d_g}$ is the embedding of nodes obtained by the *structure feature encoder*.

3) *Feature Projector*: We use a multilayer perceptron with one hidden layer to obtain the behavior and structure views, respectively, and map them into the same embedding space:

$$\mathbf{E}_f = \mathbf{W}_{p2}(\text{ReLU}(\mathbf{H}_f \mathbf{W}_{p1} + \mathbf{b}_{p1})) + \mathbf{b}_{p2}, \quad (15)$$

$$\mathbf{E}_g = \mathbf{W}_{p2}(\text{ReLU}(\mathbf{H}_g \mathbf{W}_{p1} + \mathbf{b}_{p1})) + \mathbf{b}_{p2}, \quad (16)$$

where $\mathbf{E}_f$ and $\mathbf{E}_g$ are the behavior view and structure view, respectively. The weight matrix $\mathbf{W}_{p*}$ and the bias $\mathbf{b}_{p*}$ are shared by the structure view projector and behavior view projector.

We define the contrastive learning task on each account's behavior and structure views, considering each view as an individual sample, resulting in a total of $2N$ samples.

C. Contrastive Learning Module

We use *contrastive learning module* to pre-train MLaD². It helps the model learn discriminative representations by adjusting instance pairs' distances in the latent space and reduces its dependence on the labeled training data. Specifically, we first construct contrastive instance pairs consisting of behavior and structure views. Then, we employ a contrastive loss as the objective function to guide the model in mapping

positive instance pairs to neighboring locations within the embedding space, while mapping negative instance pairs to dissimilar positions.

1) *Instance Pairs*: Given an account, we construct a positive instance pair with its behavior view $\mathbf{e}_i$ and structure view $\mathbf{e}_j$. The remaining $2(N-1)$ behavior and structure views of the other $N-1$ samples are treated as negative. We save the positive and negative pairs into their corresponding instance pair pools.

2) *Objective Function*: The objective of the *contrastive learning module* is to minimize the distance between two views of an account. Following [32], we adopt NT-Xent (the normalized temperature-scaled cross-entropy loss) as the loss function for a positive instance pair $(\mathbf{e}_i, \mathbf{e}_j)$. Let $\text{sim}(\mathbf{e}_i, \mathbf{e}_j)$ denote the cosine similarity between $\mathbf{e}_i$ and $\mathbf{e}_j$. The NT-Xent for $(\mathbf{e}_i, \mathbf{e}_j)$ is defined as:

$$l_{(i,j)} = -\log \frac{\exp(\text{sim}(\mathbf{e}_i, \mathbf{e}_j)/\tau)}{\sum_{q=1}^{2N} I(q \neq i) \exp(\text{sim}(\mathbf{e}_i, \mathbf{e}_q)/\tau)}, \quad (17)$$

where $I(q \neq i)$ is an indicator function, whose value is 1 if $q \neq i$ and 0 otherwise. τ is hyperparameter called a temperature parameter. We discuss the impact of τ 's value on the performance of MLaD² in Section IV-E. The overall loss of the *contrastive learning module* is computed across all positive pairs, including both $(\mathbf{e}_i, \mathbf{e}_j)$ and $(\mathbf{e}_j, \mathbf{e}_i)$, as follows:

$$\mathcal{L}_{SSL} = \frac{1}{2N} \sum_{q=1}^N [l(2q-1, 2q) + l(2q, 2q-1)]. \quad (18)$$

D. Money Laundering Detection Module

We employ an MLP with one hidden layer as the classifier for detecting MLAs. As the *feature projector* in the *feature encoder module* could remove some helpful information for classification, we feed the unmapped representations (i.e., $\mathbf{H}_f$ and $\mathbf{H}_g$) into the MLP. The class of accounts can be estimated as follows:

$$\hat{\mathbf{Y}} = \sigma((\text{ReLU}([\mathbf{H}_f \parallel \mathbf{H}_g] \mathbf{W}_{s1} + \mathbf{b}_{s1})) \mathbf{W}_{s2} + \mathbf{b}_{s2}), \quad (19)$$

where $\sigma(\cdot)$ is the softmax function, $\parallel$ denotes the concatenation operation, and $\hat{\mathbf{Y}}$ is the predicted class. The training objective of the MLP is to minimize the following cross-entropy loss $\mathcal{L}_{SL}$:

$$\mathcal{L}_{SL} = -\frac{1}{N_L} \sum_{i=1}^{N_L} ((1 - \mathbf{y}_i) \log(1 - \hat{\mathbf{y}}_i) + \mathbf{y}_i \log(\hat{\mathbf{y}}_i)), \quad (20)$$

where $\hat{\mathbf{y}}_i$ is the i -th value of $\hat{\mathbf{Y}}$, and $\mathbf{y}_i$ is the i -th account's ground truth.

E. Training Procedure of MLaD²

Algorithm 1 illustrates the training procedure of MLaD². Specifically, MLaD² first builds TRNNS G by calculating account similarity and assigning edge weights (lines 3-5). In the pre-training phase, it acquires the behavior embedding of nodes using the behavior feature encoder (line 8). Additionally, it obtains the graph structure embedding of nodes based

Algorithm 1 Training Procedure of MLaD²**Input:**

Transaction records, Node attribute: $\mathbf{X}$, Transaction characteristics: $\{\mathbf{M}_i\}_{i=1}^N$, Number of pre-training and fine-tuning epochs: e_p, e_f .

Output:

The well-trained MLaD².

- 1: Randomly initialize the parameters of MLaD².
- 2: # *Build the TRNNS G*.
- 3: Construct $G = (\mathcal{V}, \mathcal{E})$ based on transaction records.
- 4: Calculate $P(c_k | u_i)$ via Eq.(2)-Eq.(6).
- 5: Weight the $\mathcal{E}$ to obtain the well-built TRNNS.
- 6: # *Pre-training phase*.
- 7: **for** $i = 1, 2, \dots, e_p$ **do**
- 8: Embed transaction behavior: $\mathbf{H}_f = f(\{\mathbf{M}_i\}_{i=1}^N, \varphi)$.
- 9: Embed graph structure: $\mathbf{H}_g = g(G, \theta)$.
- 10: Map behavior embedding: $\mathbf{E}_f = \text{Projector}(\mathbf{H}_f, \zeta)$.
- 11: Map structure embedding: $\mathbf{E}_g = \text{Projector}(\mathbf{H}_g, \zeta)$.
- 12: Calculate $\mathcal{L}_{SSL}$ via Eq.(18).
- 13: $\theta, \varphi, \zeta \leftarrow \text{Adam}(\mathcal{L}_{SSL})$.
- 14: **end for**
- 15: # *Fine-tuning phase*.
- 16: **for** $i = 1, 2, \dots, e_f$ **do**
- 17: Predict class for accounts: $\hat{\mathbf{Y}} = \text{clf}([\mathbf{H}_f \| \mathbf{H}_g], \xi)$.
- 18: Calculate $\mathcal{L}_{SL}$ via Eq.(20).
- 19: $\theta, \varphi, \xi \leftarrow \text{Adam}(\mathcal{L}_{SL})$.
- 20: **end for**

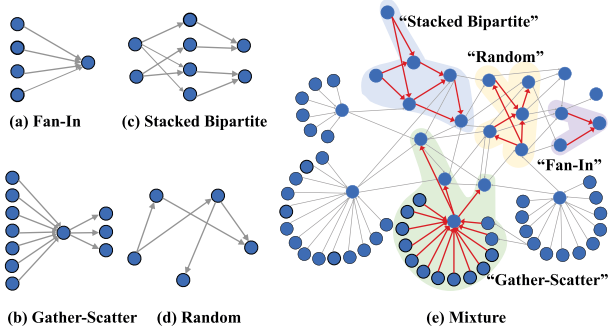


Fig. 5. Examples of ML topologies within the evaluation dataset, where nodes represent accounts and directed edges represent transaction relationships.

on G using the structure feature encoder (line 9). After that, MLaD² applies the feature projector to transform the above two views of embeddings to the same feature space (lines 10-11). Then, it performs the Adam algorithm to minimize $\mathcal{L}_{SSL}$ to optimize the feature encoder module (lines 12-13). The fine-tuning phase uses a classifier to predict the account class (line 17). Then, MLaD² computes $\mathcal{L}_{SL}$ (line 18) and fine-tunes the parameters (θ , φ , and ξ) through a backpropagation process (line 19). After e_p pre-training iterations and e_f fine-tuning iterations, we can obtain the well-trained MLaD².

IV. EXPERIMENTS AND ANALYSIS

A. Dataset

We evaluate MLaD² with a real-world dataset from a Chinese law enforcement agency. The dataset was collected

TABLE III
CHARACTERISTICS OF BASELINE METHODS

Method	Category of methods [†]			
	CL	Sup	Un-sup	Semi-sup
DCI	✓	✗	✗	✓
DGI	✓	✗	✗	✓
GAT	✗	✗	✗	✓
EvolveGCN	✗	✗	✗	✓
XGBoost	✗	✓	✗	✗
ADMLA	✗	✓	✗	✗
FlowScope	✗	✗	✓	✗
MonLAD	✗	✗	✓	✗

[†] CL=Contrastive Learning, Sup=Supervised learning, Un-sup=Unsupervised learning, Semi-sup=Semi-supervised learning.

during the investigation of an illegal online gambling case in which the operators of an illegal gambling platform attempted to make their illegal profits “clean” through ML. The dataset includes several common simple ML topologies, such as “Fan-In”, “Stacked Bipartite”, “Gather-Scatter”, and “Random” (see Figure 5(a)-5(d)). There are also more sophisticated ML patterns composed of the above simple topologies (see Figure 5(e) for an example). Figure 6 shows the density distribution diagrams of some selected statistical variables. We can see noticeable differences between MLAs and BAs regarding these variables.

We randomly selected 70%, 10%, and 20% of accounts in our dataset and used all their transaction records as the training, validation, and test data, respectively.

B. Experimental Settings

1) *Baselines*: Since we focus on the scenario where transaction records are the only data for analysis, we compared the MLaD² performance with baselines that can take transaction records as input. Among them, EvolveGCN, ADMLA, FlowScope, XGBoost, and MonLAD are recent methods for ML detection, while GAT is a representative method of GNNs for node classification tasks. Given that ML detection is a specific instance of anomaly detection, we also included two exceptional contrastive-learning-based anomaly detection methods as baselines, i.e., DCI and DGI. Table III shows the characteristics of the baselines.

- DCI¹ [34]: This method first builds an unweighted transaction network and obtains the node embeddings by analyzing their degrees in the network. Then, it splits the unweighted transaction network into k subgraphs and learns their embeddings. Finally, it applies contrastive learning on node and subgraph embeddings to detect anomalous nodes.
- DGI² [33]: This method takes the whole unweighted transaction network and a single node as instance pair for contrastive learning. The learned node embeddings are input into an MLP to detect anomalous nodes.

¹<http://github.com/wyl7/DCI-pytorch>

²<http://github.com/PetarV-/DGI>

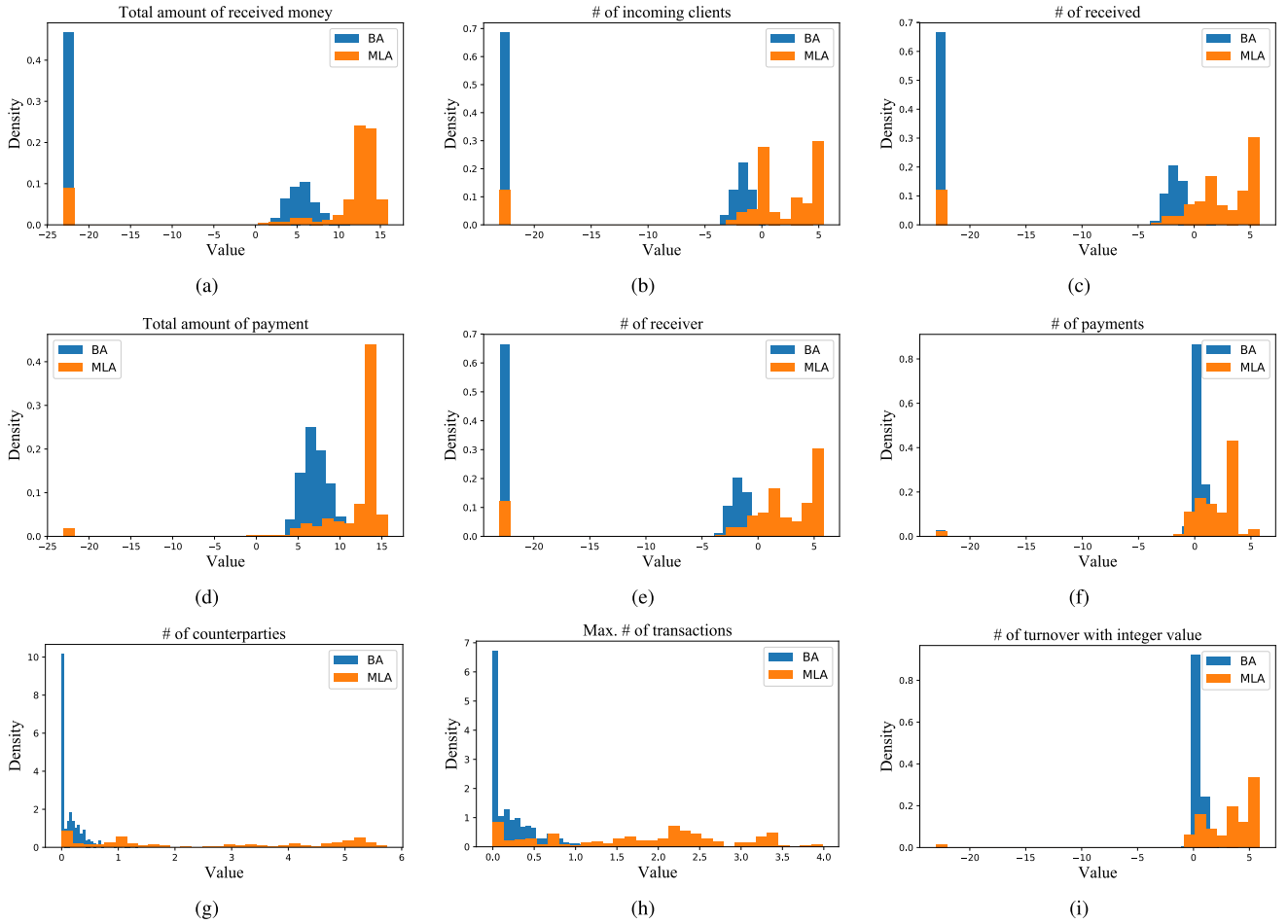


Fig. 6. The data distribution plot of selected statistical variables. The x-axis represents the variable's value, and the y-axis represents the density. Due to the significant differences in the magnitude of variables, we applied a logarithmic transformation to their original values for better visualization. We replaced the minimum value 0 with 1e-10.

- EvolveGCN [15]: This method is a semi-supervised GNN model for ML detection tasks. It aggregates neighboring information using GCN to obtain node embeddings and captures the graph sequence's dynamism using an RNN. In our implementation, we used transaction behavior statistics as the initial attributes of nodes and generated graph sequences at the granularity of days.
- GAT³ [38]: This method is also semi-supervised. It adds node-level attention compared to GCN. The node initialization method is the same as EvolveGCN.
- XGBoost [26]: This method efficiently implements the gradient boosting decision tree (GBDT). Jullum et al. [26] have designed a supervised ML detection approach based on XGBoost. However, they use user profile features as representations of accounts. Due to our focus on scenarios where only transaction records are available, we replace the input of their approach with the account representations generated by MLad².
- ADMLA [17]: This supervised method detects suspicious accounts based on transaction relationships and user profile information. As user profiles are unavailable in

our scenario, we still use the account representations generated by MLad² as its input.

- FlowScope⁴ [10]: This unsupervised method identifies MLAs by searching dense flows in a transaction network.
- MonLAD⁵ [11]: This unsupervised method takes transaction streams as input and exploits a statistical deviation of the transaction amount to detect MLAs.

2) *Evaluation Metrics*: We employ the macro average of precision, recall, F_1 -score, accuracy, and False Positive Rate (FPR) as the evaluation metrics. Previous works have widely used these metrics to assess the performance of ML detection [9], [11], [15].

3) *Implementation Details*: We trained the supervised methods using 100% labeled accounts and their transaction records in the training data (denoted as 100-LTD). We randomly selected 10%-80% accounts from the training data and assigned labels to them (denoted as 10-LTD, 20-LTD, 40-LTD, 60-LTD, and 80-LTD, respectively), utilizing their transaction records to train the semi-supervised methods. All experiments were executed on a server with Xeon 6138 CPU and an NVIDIA GeForce RTX3090 GPU. To ensure that the results

³<http://github.com/Diego999/pyGAT>

⁴<http://github.com/csqxiao/FlowScope>

⁵<http://github.com/BGT-M/MonLAD>

TABLE IV
PARAMETER SETTINGS OF MLAD² AND BASELINE METHODS

Method	Parameter Settings
DCI	# of clusters=2, # of pre-training epochs=50, # of fine-tuning epochs=150.
DGI	Input dimension=64, Node/graph vector dimension=128, # layer of GNN=2.
GAT	# of attention heads=8, Node vector dimension=32, Learning rate=0.001.
EvolveGCN	Node vector dimension=128, The other parameters are same as GAT.
XGBoost	# of estimators=10, Depth of trees=4.
ADMLA	Hidden state dimension=128
FlowScope	Imbalance cost rate=4
MonLAD	Min thresholds for effective fan-in and fan-out=1000
MLAD ²	# of pre-training epochs=1500, # of fine-tuning epochs=300, Optimizer=Adam, Learning rate=0.001, $d = 32$, $d_g = d_c = d_j = 128$, $d_{s1} = d_{s2} = 2$

were not derived by chance, we repeated each experiment ten times, and the final performance of each method was the average of the ten trials. Table IV lists the parameter settings of MLAD² and baseline methods.

C. Performance Comparisons With Existing Methods

Table V presents the performance comparison of MLAD² with supervised and semi-supervised baselines. Table VI reports the comparison results of MLAD² trained on the 10-LTD with unsupervised baselines. Based on the results, we have the following observations.

- Among contrastive learning-based baselines, MLAD² achieved a 3%-11% improvement in F_1 -score compared to DCI and a 1.64%-8.92% improvement compared to DGI. DCI and DGI used “node versus subgraph” and “graph versus graph” as instance pairs, respectively. They both calculated the average value of node embeddings to obtain the graph embedding for contrastive learning. However, the averaging operation may cause the loss of fine-grained node information, hindering the model’s ability to learn discriminative features between nodes. In contrast, MLAD² adopts “SvB” as instance pairs, which focus on the nodes’ fine-grained features and contribute to extracting inconspicuous suspicious account information.
- Among semi-supervised learning-based baselines, MLAD² outperforms EvolveGCN and GAT in terms of F_1 -score, with improvements of 1.64%-4.69% and 3.93%-4.51%, respectively. The reason is that EvolveGCN and GAT aggregate information from both MLA and BA neighbors, introducing noise to the representation of the target node and impeding the model’s classification capability. It proves the inconsistency problem is severe and considerable when applying GNNs to ML detection. On the contrary,

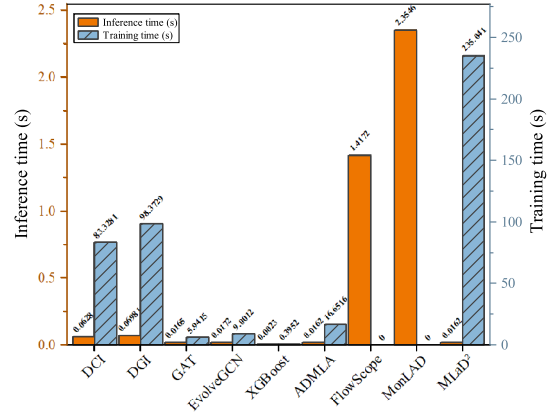


Fig. 7. Training and inference time cost comparisons with baseline methods.

MLAD² considers the similarity between neighbors and the target node during the aggregation process, reducing the noise in the transferred neighbor information.

- MLAD² exhibits superior MLA discriminative ability than supervised learning-based baselines. Compared to XGBoost and ADMLA, the F_1 -improvements are 2.71% and 3.26%, respectively. The possible reason is that, by employing CSSL on massive unlabeled data, MLAD² can learn intrinsic discriminative features between MLAs and BAs and better comprehend the data distribution.
- MLAD² achieves significant performance gains with a small quantity of labeled data compared to unsupervised learning-based baselines. When training with 10-LTD, the F_1 -score and accuracy improvements of MLAD² over the best one among FlowScope and MonLAD are 24.80% and 6.93%, respectively. These gains can be attributed to the robust ability of MLAD² to extract discriminative knowledge from the “SvB” instance pair. In contrast, FlowScope and MonLAD judge the abnormalities of accounts based on the statistical density of transaction flows and balance changes, respectively. Their strategies may miss the potential correlations and patterns among MLAs when dealing with complex transactional data.
- Figure 7 shows the training and inference time consumption of MLAD² and baseline methods. We can see that the training time of MLAD² is longer than all the baselines. However, its inference time is shorter than most of the baseline models. Compared with the best supervised learning-based baseline XGBoost and the best semi-supervised learning-based baseline DGI, MLAD² obtains 2.71% and 1.46%-8.92% improvements in F_1 -score at the cost of only 234.65s and 136.67s additional training time, respectively. Moreover, the training of MLAD² on 51,326 transactions was finished within 4 minutes, demonstrating its feasibility for practical applications.
- MLAD² keeps a zero FPR in all the comparisons in Table V and Table VI, which further demonstrate its practicality in real-world ML detection scenarios. Some other methods, such as GAT, EvolveGCN, and MonLAD, also achieve zero FPR in all comparisons. The FPRs of DCI and DGI, while not being zero, are at a low level.

TABLE V
PERFORMANCE COMPARISONS WITH SUPERVISED AND SEMI-SUPERVISED BASELINE METHODS

Method	100-LTD					80-LTD				
	Precision	Recall	F_1	Accu.	FPR	Precision	Recall	F_1	Accu.	FPR
<i>DCI</i> [34]	0.9595	0.9000	0.9216	0.9660	0.0026	0.9520	0.9025	0.9184	0.9624	0.0026
<i>DGI</i> [33]	0.9684	0.9015	0.9295	0.9761	0.0114	0.9628	0.9152	0.9356	0.9779	0.0109
<i>GAT</i> [38]	0.9874	0.8780	0.9242	0.9766	0.0000	0.9880	0.8841	0.9284	0.9778	0.0000
<i>EvolveGCN</i> [15]	0.9720	0.8707	0.9131	0.9731	0.0000	0.9720	0.8707	0.9131	0.9731	0.0000
<i>XGBoost</i> [26]	0.9721	0.8976	0.9305	0.9778	0.0000	-	-	-	-	-
<i>ADMLA</i> [17]	0.9874	0.8780	0.9241	0.9766	0.0000	-	-	-	-	-
<i>MLaD²(ours)</i>	0.9923	0.9268	0.9567	0.9859	0.0000	0.9892	0.9241	0.9538	0.9850	0.0000
gain	+0.37%	+2.53%	+2.41%	+0.70%	-	+0.12%	+0.89%	+1.82%	+0.71%	-
<i>MLaD²/w</i>	0.9887	0.9180	0.9496	0.9838	0.0000	0.9918	0.9220	0.9535	0.9850	0.0000
<i>MLaD²/ft</i>	0.9881	0.9039	0.9407	0.9813	0.0000	0.9917	0.9207	0.9527	0.9848	0.0000

Method	60-LTD					40-LTD				
	Precision	Recall	F_1	Accu.	FPR	Precision	Recall	F_1	Accu.	FPR
<i>DCI</i> [34]	0.9616	0.9031	0.9251	0.9673	0.0130	0.9526	0.8988	0.9179	0.9629	0.0104
<i>DGI</i> [33]	0.9730	0.9115	0.9387	0.9797	0.0125	0.9686	0.9107	0.9357	0.9784	0.0083
<i>GAT</i> [38]	0.9861	0.8659	0.9155	0.9742	0.0000	0.9849	0.8537	0.9066	0.9719	0.0000
<i>EvolveGCN</i> [15]	0.9653	0.8700	0.9100	0.9719	0.0000	0.9708	0.8707	0.9131	0.9742	0.0000
<i>XGBoost</i> [26]	-	-	-	-	-	-	-	-	-	-
<i>ADMLA</i> [17]	-	-	-	-	-	-	-	-	-	-
<i>MLaD²(ours)</i>	0.9921	0.9244	0.9551	0.9855	0.0000	0.9918	0.9220	0.9535	0.9850	0.0000
gain	+0.60%	+1.29%	+1.64%	+0.58%	-	+0.69%	+1.13%	+1.78%	+0.66%	-
<i>MLaD²/w</i>	0.9919	0.9220	0.9533	0.9850	0.0000	0.9891	0.9217	0.9522	0.9845	0.0000
<i>MLaD²/ft</i>	0.9902	0.9194	0.9513	0.9843	0.0000	0.9891	0.9238	0.9535	0.9849	0.0000

Method	20-LTD					10-LTD				
	Precision	Recall	F_1	Accu.	FPR	Precision	Recall	F_1	Accu.	FPR
<i>DCI</i> [34]	0.9359	0.8450	0.8786	0.9531	0.0155	0.8597	0.8309	0.8370	0.9329	0.0492
<i>DGI</i> [33]	0.9360	0.8538	0.8885	0.9641	0.0130	0.8999	0.8296	0.8578	0.9535	0.0187
<i>GAT</i> [38]	0.9849	0.8537	0.9062	0.9719	0.0000	0.9850	0.8537	0.9056	0.9719	0.0000
<i>EvolveGCN</i> [15]	0.9637	0.8700	0.9099	0.9719	0.0000	0.9439	0.8742	0.9049	0.9696	0.0000
<i>XGBoost</i> [26]	-	-	-	-	-	-	-	-	-	-
<i>ADMLA</i> [17]	-	-	-	-	-	-	-	-	-	-
<i>MLaD²(ours)</i>	0.9867	0.9183	0.9492	0.9836	0.0000	0.9876	0.9143	0.9470	0.9830	0.0000
gain	+0.18%	+4.83%	+3.93%	+1.17%	-	+0.26%	+4.01%	+4.14%	+1.11%	-
<i>MLaD²/w</i>	0.9803	0.9185	0.9467	0.9827	0.0000	0.9653	0.9242	0.9436	0.9813	0.0000
<i>MLaD²/ft</i>	0.9678	0.9052	0.9336	0.9785	0.0000	0.9417	0.8693	0.9012	0.9686	0.0000

For supervised methods, we present their results on 100-LTD.

The value in bold indicates the best score obtained by all methods, and the underlined value indicates the second best score obtained by all methods.

TABLE VI
PERFORMANCE COMPARISONS WITH UNSUPERVISED
BASELINE METHODS

Method	Precision	Recall	F_1	Accu.	FPR
<i>FlowScope</i> [10]	0.6877	0.7125	0.6990	0.8878	0.0701
<i>MonLAD</i> [11]	1.0000	0.1068	0.1930	0.9137	0.0000
<i>MLaD²(ours)</i>	0.9876	0.9143	0.9470	0.9830	0.0000
gain	-1.24%	+20.18%	+24.80%	+6.93%	-
<i>MLaD²/w</i>	0.9653	0.9242	0.9436	0.9813	0.0000
<i>MLaD²/ft</i>	0.9417	0.8693	0.9012	0.9686	0.0000

The value in bold indicates the best score obtained by all methods, and the underlined value indicates the second-best score obtained by all methods.

FlowScope exhibits the highest FPR. The possible reason is that the dense flows of money transfers identified by FlowScope may involve BAs, as MLAs often carry out transactions with BAs to disguise their activities.

In summary, MLaD² achieves the best overall detection performance compared with baseline methods. Table VII presents the p -values of paired t -test of the performance comparisons in terms of F_1 -score. The results demonstrate that the improvements of MLaD² over the baselines are statistically significant at $\alpha = 0.05$.

D. Ablation Study

To verify the effectiveness of the TRNNS in MLaD², we built a transaction network with equal edge weights and executed MLaD² on it. We refer to this variant of MLaD² as MLaD²/w. We first observed the loss variations of MLaD² and MLaD²/w during the pre-training and fine-tuning phases. Figure 8 shows the observation results. We can see that MLaD² exhibits a more stable optimization process and converges faster when using TRNNS. It reveals the effectiveness of

TABLE VII
PAIRED t -TEST RESULTS ON THE IMPROVEMENTS OF MLaD² COMPARED TO BASELINES IN TERMS OF F_1 -SCORE

Dataset	Method							
	DCI	DGI	GAT	EvolveGCN	XGBoost	ADMLA	FlowScope	MonLAD
100-LTD	<0.001	0.0018	0.0107	<0.001	<0.001	<0.001	—	—
80-LTD	<0.001	0.0114	<0.001	<0.001	—	—	—	—
60-LTD	<0.001	0.0020	0.0165	<0.001	—	—	—	—
40-LTD	<0.001	0.0010	0.0350	<0.001	—	—	—	—
20-LTD	<0.001	<0.001	0.0071	<0.001	—	—	—	—
10-LTD	<0.001	<0.001	0.0246	<0.001	—	—	<0.001	<0.001

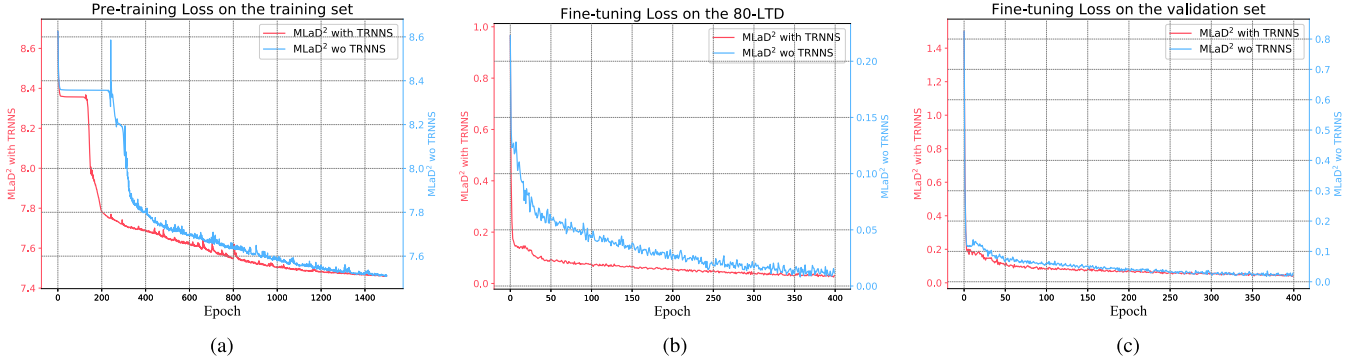


Fig. 8. The loss variation of MLaD² and MLaD²/ w during pre-training and fine-tuning phase. (a) The loss variation of MLaD² and MLaD²/ w during the pre-training phase. (b) The loss variation of MLaD² and MLaD²/ w during the fine-tuning phase on the 80-LTD. (c) The loss variation of MLaD² and MLaD²/ w during the fine-tuning phase on the validation set.

TRNNS in accelerating model training by reducing the aggregation of noise information from dissimilar neighbors.

Further, we compared the performance of MLaD²/ w with those of MLaD². The second last row in Table V reports the performance of MLaD²/ w . We can see that MLaD² has an average F_1 -score gain of approximately 0.25% across various label fractions of training data compared with MLaD²/ w , indicating that assigning higher weights to information from similar neighbors improves detection performance. The improvement is minor but significant because the number of accounts can be tens of thousands.

To validate the effectiveness of fine-tuning in MLaD², we remove the fine-tuning phase from MLaD² and test its performance. We refer to this variant as MLaD²/ ft . The last row in Table V presents the performance results. Evidently, the fine-tuning phase can improve the model's overall performance on the training data with different proportions of labeled samples, especially when the number of labeled samples is small. For example, MLaD² improves by 4.58% in F_1 -score on 10-LTD compared with MLaD²/ ft .

Furthermore, as shown in Table V, MLaD²/ ft achieves competitive performance compared to MLaD² and outperforms almost all baselines. The results indicate that during the pre-training phase, the feature encoders learn discriminative knowledge from instance pairs rather than relying on labels, reducing MLaD²'s dependency on labeled data.

E. Parameter Sensitivity Analysis

Finally, we investigate the impacts of three hyperparameters on the performance of MLaD², i.e., the training batch size, the temperature τ in $\mathcal{L}_{SSL}$, and the number of GNN layers l .

1) *Effect of the Batch Size*: We discontinuously varied the training batch size from 128 to 2131 (i.e., the total number of samples in the training set). Figure 9 (a) shows the F_1 -score performance changes of MLaD² using different batch sizes. As we can see, when the number of labeled training data is small (e.g., 10%), larger batch sizes have a significant advantage over smaller ones. A larger batch size can provide more negative samples to facilitate the feature encoder's learning of helpful representation. When there is more labeled data, the performance improvement obtained with increasing batch size is no longer apparent. However, using a larger batch size can still get a better performance. Therefore, we finally chose 2,131 as the batch size.

2) *Effect of the Temperature*: We further analyze the significance of τ in $\mathcal{L}_{SSL}$ of MLaD². Figure 9 (b) shows the F_1 -score performance changes with different values of τ . We can see that MLaD² can obtain a relatively good detection result when $\tau = 0.1$. It can attribute to the role of τ in controlling the penalty strength on hard negative samples. Specifically, MLaD² with a small τ value tends to penalize hard negative samples more. Conversely, the model with a larger τ value is less sensitive to hard negative samples [32]. We finally set τ as 0.1.

3) *Effect of the Number of GNN Layers*: We vary l from 1 to 4 to explore the sensitivity of MLaD² to it. Figure 9 (c) shows the impact of l when MLaD² trains on various proportions of labeled training data. As we can see, the F_1 -score is relatively worse when l is small ($l = 1$) due to insufficient neighboring information. As l increases, we can observe that the F_1 -score first increases and then declines. A larger value of l indicates that a node can aggregate information from

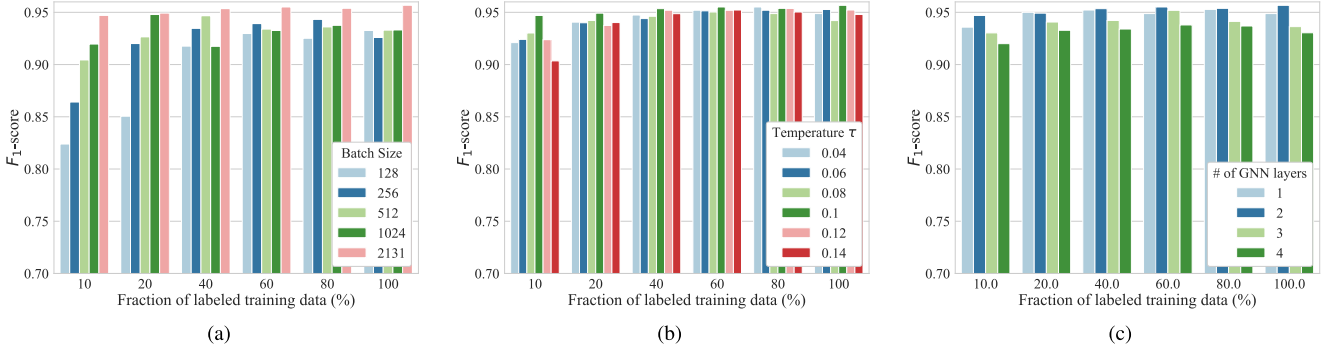


Fig. 9. Impact of hyper-parameters on the performance of MLaD² in terms of F_1 -score. (a) Impact of the training batch size. (b) Impact of τ . (c) Impact of l .

higher-order neighbors. The results suggest that neighbors farther than two orders may introduce more noise to the node representations learned by MLaD². We finally set l as 2.

F. Model Robustness

To explore MLaD²'s robustness against evolving ML strategies that malicious actors may employ to evade detection, we evaluated its performance in a realistic attack scenario. The attack scenario assumes that: (1) an attacker can infer that MLaD² utilizes a GNN-based approach, but they cannot obtain the specific design and parameters of the model; (2) the attacker has control over a set of accounts and can evade detection by manipulating transactions between the accounts.

Based on the above assumptions, we employed a basic GCN architecture [45] as the surrogate model S_{att} that attackers may build to generate adversarial samples. Then, we leveraged the Fast Gradient Sign Method (FGSM) [46] to generate adversarial samples for S_{att} . FGSM is a simple and effective technique for generating adversarial samples to attack deep learning models. The basic idea behind FGSM is to add a perturbation in the direction of the gradient of the loss with respect to the input, causing the model to make incorrect predictions. The resulting new sample is very similar to the original but is crafted to cause the model to make a misclassification.

We selected 81 MLAs in our test dataset whose transaction relationship can form a connected component in TRNNS. These MLAs are assumed to be under the attacker's control, and their data are input into FGSM. The generated adversarial samples are considered novel ML strategies. We replaced the original data of the 81 MLAs with the adversarial samples and used the tampered test data for to evaluate the robustness of MLaD². Several studies have demonstrated that adversarial samples generated for one deep learning model may also be misclassified by another model [47].

Let $\mathcal{D}_{ot}$ and $\mathcal{D}_{tt}$ denote the original and tampered test data, respectively. Table VIII reports the performance comparison of MLaD² and XGBoost (i.e., the best baseline trained with 100-LTD in terms of F_1) on $\mathcal{D}_{ot}$ and $\mathcal{D}_{tt}$. We can see that MLaD²'s performance shows a slight decrease on $\mathcal{D}_{tt}$ compared with its performance on $\mathcal{D}_{ot}$. The F_1 score of MLaD² trained with different proportions of labeled data drops 0.027-0.0687 on $\mathcal{D}_{tt}$. In contrast, XGBoost exhibits a significant decrease of 0.5972 in F_1 on $\mathcal{D}_{tt}$ compared to its F_1 on $\mathcal{D}_{ot}$. The results

reveal the robustness of MLaD² against adversarial samples (i.e., ML strategies to evade detection). Its robustness may be attributed to the fine-grained discriminative information learned from the "SvB" instance pair.

G. Case Study on Model Interpretability

The interpretability of MLaD² decisions is crucial for its trust and transparency in financial applications. To gain deeper insight into the decision-making mechanism of MLaD², we attempted to explain the prediction results of MLaD² using PGM-Explainer. PGM-Explainer [48] is an interpretability method using probabilistic graphic models to elucidate the decisions made by GNN. By inputting the prediction results of MLaD² into PGM-Explainer, we can identify those samples that are most crucial for determining the category of a particular node based on the output explanation.

We select two test samples that MLaD² has correctly classified for a case study. One sample is a BA (referred to as v_1), and the other is an MLA (referred to as v_2). Figure 10 shows the two samples and their 1-order neighbors. The red dashed circles point out the neighbors most crucial for determining the categories of v_1 and v_2 . We can see that although the neighbors of v_1 and v_2 include samples of different categories, those determining their categories are all of the same category as them due to TRNNS's effect. We also computed the cosine similarity between the vector representations of v_1 (v_2) and all its neighbors. Figure 10 lists the respective five neighbors most similar to v_1 and v_2 . It can be seen that those neighbors determining the categories of v_1 and v_2 are the most similar to them. MLaD² can find these samples by computing the similarities between the target node and their neighbors.

The above analysis provides an approximate explanation of the decision-making process of MLaD².

V. LIMITATIONS

One obvious limitation of this work is that MLaD² was evaluated on a single dataset, and the evaluation dataset is small in scale and a bit dated. Therefore, the performance of MLaD² is not guaranteed to generalize to other datasets. Moreover, the scalability of MLaD² on large datasets or real-time detection scenarios remains to be determined. Another limitation is that the interpretability of MLaD² needs to be improved. Although

TABLE VIII
PERFORMANCE COMPARISONS ON ORIGINAL AND TAMPERED TEST DATA

Dataset	Method	100-LTD					80-LTD				
		Precision	Recall	F_1	Accu.	FPR	Precision	Recall	F_1	Accu.	FPR
$\mathcal{D}_{tt}$	MLaD ²	0.9348	0.9250	0.9246	0.9250	0.0000	0.9310	0.9200	0.9195	0.9200	0.0000
	XGBoost	0.2500	0.5000	0.3333	0.5000	0.0000	-	-	-	-	-
$\mathcal{D}_{ot}$	MLaD ²	0.9923	0.9268	0.9567	0.9859	0.0000	0.9892	0.9241	0.9538	0.9850	0.0000
	XGBoost	0.9721	0.8976	0.9305	0.9778	0.0000	-	-	-	-	-

Dataset	Method	60-LTD					40-LTD				
		Precision	Recall	F_1	Accu.	FPR	Precision	Recall	F_1	Accu.	FPR
$\mathcal{D}_{tt}$	MLaD ²	0.9202	0.9050	0.9041	0.9050	0.0000	0.9329	0.9225	0.9220	0.9225	0.0000
	XGBoost	-	-	-	-	-	-	-	-	-	-
$\mathcal{D}_{ot}$	MLaD ²	0.9921	0.9244	0.9551	0.9855	0.0000	0.9918	0.9220	0.9535	0.9850	0.0000
	XGBoost	-	-	-	-	-	-	-	-	-	-

Dataset	Method	20-LTD					10-LTD				
		Precision	Recall	F_1	Accu.	FPR	Precision	Recall	F_1	Accu.	FPR
$\mathcal{D}_{tt}$	MLaD ²	0.9171	0.9050	0.9043	0.9050	0.0200	0.9049	0.8800	0.8776	0.8800	0.0060
	XGBoost	-	-	-	-	-	-	-	-	-	-
$\mathcal{D}_{ot}$	MLaD ²	0.9867	0.9183	0.9492	0.9836	0.0000	0.9876	0.9143	0.9470	0.9830	0.0000
	XGBoost	-	-	-	-	-	-	-	-	-	-

$\mathcal{D}_{ot}$ and $\mathcal{D}_{tt}$ denote the original and tampered test data, respectively.

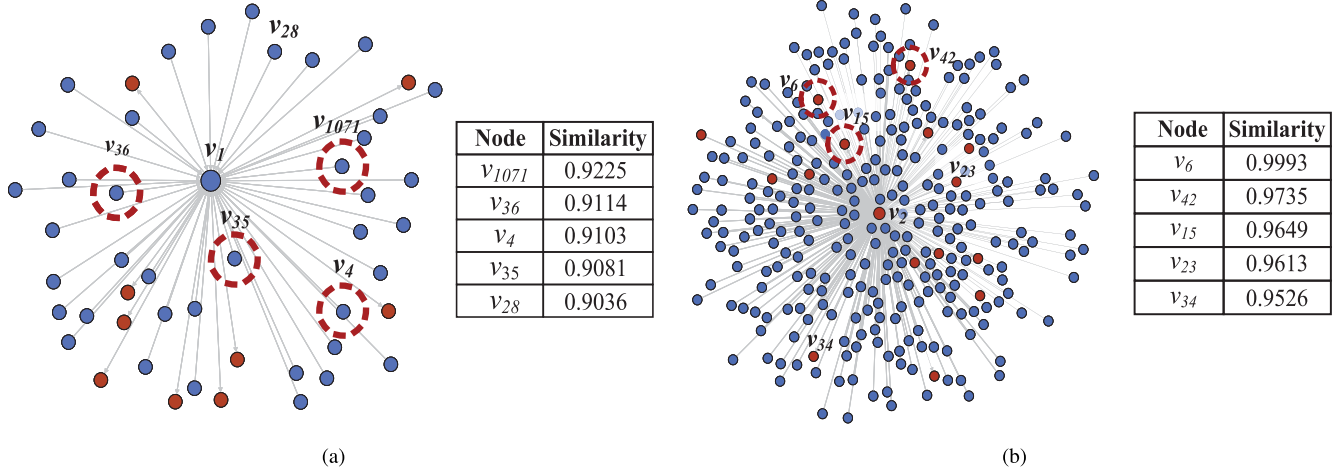


Fig. 10. Prediction result explanations of (a) a BA and (b) an MLA. The blue nodes represent BAs, and the red nodes represent MLAs. The dashed red circles highlight those samples that are most crucial for determining the category of the central node. The tables to the right of each subgraph enumerate the five neighbors most similar to the central node and their respective cosine similarities with the central node.

we have explored its interpretability using PGM-Explainer, MLaD² has weak inherent interpretability. In our future work, we will collect more extensive and real-time labeled data to train and evaluate MLaD². We also will design an inherent interpretable mechanism for it.

Users of MLaD² can utilize a small set of labeled historical data to train the framework to detect MLAs. However, there are two limitations of MLaD² in daily use. First, as MLaD² is a semi-supervised-based framework, users need to perform its training process periodically to detect MLAs in new coming data. Second, if an MLA only conducts transactions with very few accomplices and almost all of their counter-parties are BAs, MLaD² may not pick out the MLA from BAs. However, most MLAs are unlikely to engage in transactions with numerous benign accounts due to the labor and resources costs. Therefore, our method can work for most real-world scenarios.

VI. CONCLUSION

This study proposes MLaD², a semi-supervised framework based on decoupling training, for MLA detection. It overcomes the drawback of the homogeneity hypothesis of GNN in ML detection scenarios via the weighting mechanism of the proposed TRNNS. Additionally, the model successfully captures the discriminative features of accounts using CSSL based on the newly designed “SvB” instance pair, which mitigates the model’s reliance on labeled information and the adverse effects of class imbalance. Extensive experiments on a real-world dataset demonstrate that MLaD² achieves superior detection performance with a small quantity of labeled training data. Besides, the ablation study verifies the effectiveness of TRNNS and the importance of the fine-tuning process in enhancing model performance. Experimental results also proved that MLaD² has robustness against adversarial samples.

REFERENCES

- [1] A. Salehi, M. Ghazanfari, and M. Fathian, "Data mining techniques for anti money laundering," *Int. J. Appl. Eng. Res.*, vol. 12, no. 20, pp. 10084–10094, 2017.
- [2] S. N. F. S. M. Nazri, S. Zolkafli, and N. Omar, "Mitigating financial leakages through effective money laundering investigation," *Managerial Auditing J.*, vol. 34, no. 2, pp. 189–207, Feb. 2019.
- [3] A. Verhage, "Between the hammer and the anvil? The anti-money laundering-complex and its interactions with the compliance industry," *Crime, Law Social Change*, vol. 52, no. 1, pp. 9–32, Jul. 2009.
- [4] S. Gao, D. Xu, H. Wang, and P. Green, "Knowledge-based anti-money laundering: A software agent bank application," *J. Knowl. Manage.*, vol. 13, no. 2, pp. 63–75, Apr. 2009.
- [5] D. Savage, Q. Wang, X. Zhang, P. Chou, and X. Yu, "Detection of money laundering groups: Supervised learning on small networks," in *Proc. Workshops 31st AAAI Conf. Artif. Intell.*, 2017, pp. 43–49.
- [6] Z. Chen, L. D. Van Khoa, E. N. Teoh, A. Nazir, E. K. Karupiah, and K. S. Lam, "Machine learning techniques for anti-money laundering (AML) solutions in suspicious transaction detection: A review," *Knowl. Inf. Syst.*, vol. 57, no. 2, pp. 245–285, Nov. 2018.
- [7] L. Keyan and Y. Tingting, "An improved support-vector network model for anti-money laundering," in *Proc. 5th Int. Conf. Manage. E-Commerce E-Government*, Nov. 2011, pp. 193–196.
- [8] L.-T. Lv, N. Ji, and J.-L. Zhang, "A RBF neural network model for anti-money laundering," in *Proc. Int. Conf. Wavelet Anal. Pattern Recognit.*, Aug. 2008, pp. 209–215.
- [9] Z. Chen, W. M. Soliman, A. Nazir, and M. Shorfuzzaman, "Variational autoencoders and wasserstein generative adversarial networks for improving the anti-money laundering process," *IEEE Access*, vol. 9, pp. 83762–83785, 2021.
- [10] X. Li et al., "FlowScope: Spotting money laundering based on graphs," in *Proc. AAAI Conf. Artif. Intell.*, 2020, vol. 34, no. 4, pp. 4731–4738.
- [11] X. Sun et al., "MonLAD: Money laundering agents detection in transaction streams," in *Proc. 15th ACM Int. Conf. Web Search Data Mining*, Feb. 2022, pp. 976–986.
- [12] M.-C. Lee et al., "AutoAudit: Mining accounting and time-evolving graphs," in *Proc. IEEE Int. Conf. Big Data (Big Data)*, Dec. 2020, pp. 950–956.
- [13] K. Veeramachaneni, I. Arnaldo, V. Korrapati, C. Bassias, and K. Li, "AI²: Training a big data machine to defend," in *Proc. IEEE IEEE 2nd Int. Conf. Big Data Secur. Cloud (BigDataSecurity) Int. Conf. High Perform. Smart Comput. (HPSC), IEEE Int. Conf. Intell. Data Secur. (IDS)*, Apr. 2016, pp. 49–54.
- [14] D. Labanca, L. Primerano, M. Markland-Montgomery, M. Polino, M. Carminati, and S. Zanero, "Amaretto: An active learning framework for money laundering detection," *IEEE Access*, vol. 10, pp. 41720–41739, 2022.
- [15] M. Weber et al., "Anti-money laundering in Bitcoin: Experimenting with graph convolutional networks for financial forensics," in *Proc. KDD Workshop Anomaly Detection Finance*, 2019, p. 8.
- [16] A. Pareja et al., "EvolveGCN: Evolving graph convolutional networks for dynamic graphs," in *Proc. AAAI Conf. Artif. Intell.*, Apr. 2020, vol. 34, no. 4, pp. 5363–5370.
- [17] Y. Zhou et al., "Analyzing and detecting money-laundering accounts in online social networks," *IEEE Netw.*, vol. 32, no. 3, pp. 115–121, May 2018.
- [18] Y. Xie, Z. Xu, J. Zhang, Z. Wang, and S. Ji, "Self-supervised learning of graph neural networks: A unified review," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 2, pp. 2412–2429, Feb. 2023.
- [19] M. Alkhalili, M. H. Qutout, and F. Almasalha, "Investigation of applying machine learning for watch-list filtering in anti-money laundering," *IEEE Access*, vol. 9, pp. 18481–18496, 2021.
- [20] Z. Gu, D. Lin, and J. Wu, "On-chain analysis-based detection of abnormal transaction amount on cryptocurrency exchanges," *Phys. A, Stat. Mech. Appl.*, vol. 604, Oct. 2022, Art. no. 127799.
- [21] R. I. T. Jensen and A. Iosifidis, "Qualifying and raising anti-money laundering alarms with deep learning," *Expert Syst. Appl.*, vol. 214, Mar. 2023, Art. no. 119037.
- [22] U. G. Ketenci, T. Kurt, S. Önal, C. Erbil, S. Aktürkoglu, and H. S. İlhan, "A time-frequency based suspicious activity detection for anti-money laundering," *IEEE Access*, vol. 9, pp. 59957–59967, 2021, doi: 10.1109/ACCESS.2021.3072114.
- [23] C. R. Alexandre and J. Balsa, "Incorporating machine learning and a risk-based strategy in an anti-money laundering multiagent system," *Expert Syst. Appl.*, vol. 217, May 2023, Art. no. 119500.
- [24] J.-D.-J. Rocha-Salazar, M.-J. Segovia-Vargas, and M.-D.-M. Camacho-Miñano, "Money laundering and terrorism financing detection using neural networks and an abnormality indicator," *Expert Syst. Appl.*, vol. 169, May 2021, Art. no. 114470.
- [25] B. Dumitrescu, A. Baltoiu, and S. Budulan, "Anomaly detection in graphs of bank transactions for anti money laundering applications," *IEEE Access*, vol. 10, pp. 47699–47714, 2022.
- [26] M. Jullum, A. Løland, R. B. Huseby, G. Ånonsen, and J. Lorentzen, "Detecting money laundering transactions with machine learning," *J. Money Laundering Control*, vol. 23, no. 1, pp. 173–186, Jan. 2020.
- [27] J. Rissanen, "Modeling by shortest data description," *Automatica*, vol. 14, no. 5, pp. 465–471, 1978.
- [28] H. S. Assumpção, F. Souza, L. L. Campos, V. T. de Castro Pires, P. M. L. de Almeida, and F. Murai, "DELATOR: Money laundering detection via multi-task learning on large transaction graphs," in *Proc. IEEE Int. Conf. Big Data (Big Data)*, Dec. 2022, pp. 709–714.
- [29] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. NIPS*, 2017, pp. 1025–1035.
- [30] B. Kang et al., "Decoupling representation and classifier for long-tailed recognition," in *Proc. 8th Int. Conf. Learn. Represent. (ICLR)*, 2020, pp. 1–16.
- [31] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick, "Momentum contrast for unsupervised visual representation learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 9729–9738.
- [32] T. Chen, S. Kornblith, M. Norouzi, and G. E. Hinton, "A simple framework for contrastive learning of visual representations," in *Proc. 37th Int. Conf. Mach. Learn.*, vol. 119, 2020, pp. 1597–1607.
- [33] P. Veličković, W. Fedus, W. L. Hamilton, P. Liò, Y. Bengio, and R. D. Hjelm, "Deep graph infomax," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2019, pp. 1–17.
- [34] Y. Wang, J. Zhang, S. Guo, H. Yin, C. Li, and H. Chen, "Decoupling representation learning and classification for GNN-based anomaly detection," in *Proc. 44th Int. ACM SIGIR Conf. Res. Develop. Inf. Retr.*, Jul. 2021, pp. 1239–1248.
- [35] W. Weng Lo, G. K. Kulatilleke, M. Sarhan, S. Layeghy, and M. Portmann, "Inspection-L: Self-supervised GNN node embeddings for money laundering detection in Bitcoin," 2022, *arXiv:2203.10465*.
- [36] J. Zhou, C. Hu, J. Chi, J. Wu, M. Shen, and Q. Xuan, "Behavior-aware account de-anonymization on Ethereum interaction graph," *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 3433–3448, 2022.
- [37] X. Luo, X. Han, W. Zuo, Z. Xu, Z. Wang, and X. Wu, "A dynamic transaction pattern aggregation neural network for money laundering detection," in *Proc. IEEE Int. Conf. Trust, Secur. Privacy Comput. Commun. (TrustCom)*, Dec. 2022, pp. 818–826.
- [38] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph attention networks," in *Proc. 6th Int. Conf. Learn. Represent.*, 2018, pp. 1–12.
- [39] B. Knyazev, G. W. Taylor, and M. Amer, "Understanding attention and generalization in graph neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 4202–4212.
- [40] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997.
- [41] K. Cho, B. van Merriënboer, D. Bahdanau, and Y. Bengio, "On the properties of neural machine translation: Encoder-decoder approaches," 2014, *arXiv:1409.1259*.
- [42] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 6000–6010.
- [43] S. Li, "Enhancing the locality and breaking the memory bottleneck of transformer on time series forecasting," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 5243–5253.
- [44] Y. Liu, Z. Li, S. Pan, C. Gong, C. Zhou, and G. Karypis, "Anomaly detection on attributed networks via contrastive self-supervised learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 6, pp. 2378–2392, Jun. 2022.
- [45] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. 5th Int. Conf. Learn. Represent.*, 2017, pp. 1–14.
- [46] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," in *Proc. ICLR*, 2015, pp. 1–11.
- [47] Y. Liu, X. Chen, C. Liu, and D. Song, "Delving into transferable adversarial examples and black-box attacks," 2016, *arXiv:1611.02770*.
- [48] M. Vu and M. T. Thai, "PGM-explainer: Probabilistic graphical model explanations for graph neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 12225–12235.



Xuejiao Luo is currently pursuing the master's degree in computer science and technology with Shandong Computer Science Center (National Supercomputer Center, Jinan), Qilu University of Technology (Shandong Academy of Sciences), China. Her current research interests are in computer forensics and data mining.



Xiaoming Wu received the master's degree in computer science and technology from Shandong University and the Ph.D. degree in computer science and technology from Shandong University of Science and Technology. He is currently a Professor with Shandong Computer Science Center (National Supercomputer Center in Jinan), Qilu University of Technology (Shandong Academy of Sciences), China. His current research interests include industrial internet, big data analysis, and information security. He was selected as a young and middle-aged expert with outstanding contributions in Shandong Province.



Xiaohui Han (Member, IEEE) received the B.Eng. and Ph.D. degrees in computer science and technology from Shandong University, China, in 2006 and 2012, respectively. From 2011 to 2012, he was a Visiting Scholar with the Artificial Intelligence Laboratory, MIS Department, The University of Arizona, Tucson, AZ, USA. He is currently a Professor with Shandong Computer Science Center (National Supercomputer Center in Jinan), Qilu University of Technology (Shandong Academy of Sciences), China. He has coauthored more than 30 peer-reviewed articles. His research interests include cybersecurity, digital forensics, data mining, and machine learning. He serves as a PC member for several international conferences.



Wenbo Zuo received the master's and Ph.D. degrees in astrophysics from the University of Chinese Academy of Sciences. He is currently an Assistant Researcher with Shandong Computer Science Center (National Supercomputer Center in Jinan), Qilu University of Technology (Shandong Academy of Sciences), China. His current research interests include cyber security, data mining, and machine learning.



Wenyin Liu (Senior Member, IEEE) received the B.Eng. and M.Eng. degrees in computer science from Tsinghua University, Beijing, and the D.Sc. degree from the Technion—Israel Institute of Technology, Haifa. He was a Professor with the School of Computer Science and Technology and the Director of the Web Identity Security Laboratory, Guangdong University of Technology, the Deputy Director of the Multimedia Software Engineering Research Centre, City University of Hong Kong, from 2013 to 2016, an Assistant Professor with the Department of Computer Science, City University of Hong Kong, from 2002 to 2012, and a full-time Researcher at Microsoft Research China/Asia from 1999 to 2001. He is currently a full-time Researcher with Zhongguancun Laboratory. His research interests include anti-phishing and web identity authentication and management. He is a fellow of IAPR. In 2003, he was awarded the International Conference on Document Analysis and Recognition Outstanding Young Researcher Award from the International Association for Pattern Recognition (IAPR). He was the TC10 Chair of IAPR from 2006 to 2010.